Industrial-Grade Unit Testing and CI/CD for OpenUnderstand

Parsa barikbin

Homework 3 — Advanced Software Testing and Program Analysis

Student ID: 404131058 — Branch: feature/testing-404131058

Table of Contents

1. Introduction

OpenUnderstand is an open-source re-implementation of the SciTools *Understand* Python API for static analysis of Java programs. It models the same core abstractions as the commercial tool — entities, references, lexemes, and the static-analysis relationships between them — and exposes them through a Python facade (`Db`, `Ent`, `Kind`, `Ref`) backed by a peewee/SQLite database.

This report documents an industrial-grade quality-engineering effort on that project: a maintainable unit-testing architecture, a GitHub Actions CI/CD pipeline with enforced quality gates, automated test generation with Pynguin, coverage and mutation analysis, oracle-based validation against the commercial tool, and the discovery and professional reporting of real faults.

The unit chosen for testing in isolation is the public API module `openunderstand/oudb/api.py`, focusing on **entity kinds** and **reference kinds** through the `Kind` class and on the `Ref`/`Ent`/`Db` objects that surface them. Reference kinds (`Java Call`/`Java Callby`, plus the `Define`/`Contain` families) were selected because they directly exercise the two most interesting relationships the assignment asks about: *inverse references* and *parent–child structure*.

2. Testing Architecture

2.1 Isolation strategy

Importing `openunderstand.oudb.api` normally pulls in the entire ANTLR/javalang Java-parsing stack: almost every metric module imports `openunderstand.ounderstand.project`, which in turn builds the generated ANTLR parsers. None of that is needed to test the database-backed api/db layer, and it would make "unit" tests slow, brittle, and non-isolated.

The suite therefore installs a :pep:`302` **meta-path finder** that synthesises stub modules for those prefixes before importing the api. The unit under test is thus decoupled from its heavy collaborators — a textbook application of the test-double/seam technique.

The finder lives in a single module, `tests/_isolation.py`, with two callers: `tests/conftest.py` for the pytest suite, and `tests/_pynguin_support/sitecustomize.py` for Pynguin, which runs in a separate process *and* virtualenv and so loads it by path. One implementation means both paths isolate exactly the same surface.

Two details in that finder are worth calling out because both were driven by real failures:

- Stub modules set `__path__ = []` so they count as *packages* and nested imports (`openunderstand.metrics.cyclomatic`) resolve recursively.

- The module-level `__getattr__` synthesises arbitrary names but **refuses dunders**. Introspection tools walk `sys.modules` probing `__file__`, `__all__` and similar, expecting a real value or `AttributeError`; returning a class instead made Hypothesis' constant-collection pass crash with `TypeError: argument of type 'type' is not iterable`.

Each test runs against a fresh **in-memory SQLite** database (peewee `:memory:`), created and dropped per test, so the suite is hermetic, fast, and order-independent, and never touches a developer's real `.oudb` files.

2.2 Layout

```
tests/
├── _isolation.py             # the stub meta-path finder (one implementation)
├── conftest.py               # isolation bootstrap + shared fixtures
├── unit/                     # example-based tests (71)
│   ├── test_kind.py          #   entity & reference kinds, inverse reference
s
│   ├── test_ref.py           #   reference objects
│   ├── test_entity_and_db.py #   parent-child, unknown entities, db queries
│   └── test_misc.py          #   remaining in-scope accessors / dunders
├── property/                 # Hypothesis property-based tests (23)
│   └── test_api_properties.py
├── _pynguin_support/         # sitecustomize shim for Pynguin
└── generated/                # curated Pynguin output
```

Both tiers exercise the **same** unit in isolation and differ only in how inputs are chosen: hand-picked examples versus generated ones. Splitting them keeps the example-based suite readable while letting the property tier be selected or skipped independently (`pytest -m property`).

Fixtures provide a realistic slice of the Java kind table (with inverse `_inv` wiring identical to `oudb/fill.py`), a `File → Class → Method → Parameter` entity tree, a `Java Call` reference, and an `api.Db` bound to the in-memory database. Factory fixtures (`make_kind`/`make_ent`/`make_ref`) wrap ORM rows as api dataclasses exactly as the production code does — re-fetching by primary key first, because `Model.create()` only records explicitly-set columns in peewee's `__data__` and the dataclasses require every field.

3. Manual Unit Testing

Handcrafted tests cover the eight behaviours required by Part 3:

#	Requirement	Where	Status
1	Handcrafted unit tests	all `tests/unit/test_*.py`	done

#	Requirement	Where	Status
2	Normal behaviour	name/longname/check/list_*/accessors	done
3	Malformed input	reference to a non-existent entity id (`test_ref`)	scoped — see below
4	Edge cases	empty filter string, no-match fallback (`test_kind`)	done
5	Unresolved/unknown entities	`ent_from_id(unknown) → None`; unknown kind filter → empty	done
6	Inverse references	`Kind.inv()` (entity-kind raises; reference-kind via xfail)	done
7	Parent–child relationships	full `parent()` chain walk (`test_entity_and_db`)	done
8	Multi-pass analysis	parser-stage concern; documented + skipped placeholder	not implemented — see below

Two requirements are **deliberately not implemented**, and the reasoning is the same for both: the unit under test is the api/db layer, and the parser is a stubbed collaborator.

- **Requirement 3 (malformed Java snippets).** Malformed *Java* can only be observed by the ANTLR grammar and the listener passes. At the api layer the analogous defect is a malformed *database row*, which is what `test_ent_for_unknown_id_raises_does_not_exist` covers: a reference pointing at an entity id that does not exist must raise rather than silently pass. Testing the grammar itself would be an integration test against a different unit.
- **Requirement 8 (multi-pass analysis).** The api layer performs no multi-pass analysis; declaration and reference-resolution passes belong to `openunderstand.ounderstand`. The requirement is qualified "where applicable" and it does not apply to this unit. It is marked with an explicit `skip` carrying that reason so the decision is recorded in the test output itself, and is addressed instead by oracle-based/differential testing (Section 6).

Scoping a unit and then testing everything inside it is a different activity from testing whatever happens to be reachable. Both decisions are recorded in the code, not only in this report.

Three behaviours are encoded as `xfail` tests because they expose genuine bugs (Section 7); marking them as expected failures keeps the pipeline green while still documenting the defects in executable form.

3.1 Property-based testing (bonus)

`tests/property/` adds 23 Hypothesis tests over the same unit. Where the example-based tier pins behaviour at inputs the author thought of, these state *invariants* and let Hypothesis search for counterexamples:

Invariant	Why it matters
`Kind.check(s)` ⟺ case-insensitive substring	the filter primitive every `kindstring` query is built on
`Ent.simplename()` never contains `.`	the documented Java contract
`a == b` ⟹ `hash(a) == hash(b)`	`Db.ents()` returns a `set`; a violation silently drops entities
`__eq__` is symmetric	required of any `__eq__`; catches `==` → `<=` style defects
`value()`/`type()` return `None` only for `None`	null-propagation contract
relativising a path preserves its basename	**falsified — became Fault #5**

The last row is the payoff: within a few dozen generated examples Hypothesis found that `Db.relative_file_name()` returns a path escaping the project root (Section 7). No hand-written example in the suite had come close.

One property was *deliberately weakened* after Hypothesis falsified it: case-insensitive matching is only asserted over ASCII, because `'ı'.upper() == 'I'` makes `str.lower()` case folding non-round-trip-safe for Unicode. That is correct Python behaviour rather than a defect, so it is recorded as a documented limitation of kind filtering instead of a bug.

4. Automated Test Generation (Pynguin)

Pynguin 0.45.0 (DYNAMOSA, `--seed 42`, `--assertion-generation MUTATION_ANALYSIS`) targets `openunderstand.oudb.api`. Because Pynguin pins old transitive dependencies, it runs in its own virtual environment; the `scripts/run_pynguin.*` wrappers set `PYNGUIN_DANGER_AWARE` (Pynguin *executes* the module under test, so the acknowledgement is mandatory) and put the isolation shim on `PYTHONPATH` so generation focuses on the api/db layer rather than the parser.

4.1 Getting Pynguin to run at all

Generation initially failed, and the two failures are worth reporting because diagnosing them is most of the engineering content of this part.

Failure 1 — `RecursionError: maximum recursion depth exceeded`. Pynguin snapshots module state with `dill`, which recursively pickles every object reachable from the target module. `api.py` reaches the peewee ORM classes, whose metaclass and `ForeignKeyField` back-reference graphs are deeply self-referential, overflowing CPython's default 1000-frame limit. *Fixed* by raising the recursion limit to 30 000 in the Pynguin start-up shim (`tests/_pynguin_support/sitecustomize.py`).

Failure 2 — `NameError: name 'IterableClassWatcher' is not defined`. This was initially recorded as an unfixable internal Pynguin bug. That diagnosis was wrong, and the correction is instructive: **`IterableClassWatcher` is not a Pynguin symbol at all.** It is a metaclass in *GitPython* (`git/util.py`), the deprecated `Iterable` shim that GitPython 3.1.x warns about. The full mechanism:

1. Pynguin runs its search in a `multiprocess` worker and ships module state to it with `dill`.
2. `dill._create_type` reconstructs `git.util.Iterable`.
3. Constructing that class invokes its metaclass, whose `__init__` refers to its own name as a *module global*:

   ```
   class IterableClassWatcher(type):
       def __init__(cls, name, bases, clsdict):
           for base in bases:
               if type(base) == IterableClassWatcher:   # global lookup
   ```

4. During reconstruction that global is not yet bound in the rebuilt namespace, so the worker dies with `NameError`.

`api.py` imports `git` at module level but uses it in exactly one function, `update_db()`, which needs a real repository and is already marked `# pragma: no cover` — i.e. it is outside the unit under test. **The fix was therefore to add `git` to the isolation shim's stubbed prefixes**, keeping GitPython out of the module graph entirely. Two lines of configuration, once the cause was actually identified.

A third, smaller failure followed (`ModuleNotFoundError: _ou_test_isolation`): the shim loaded `tests/_isolation.py` under a synthetic module name without registering it in `sys.modules`, and Pynguin re-imports the `__module__` of everything it reaches in order to parse its syntax tree. Registering the module before `exec_module` resolved it.

Lesson. "Internal tool bug" is a diagnosis worth double-checking. The symbol in the traceback belonged to a *transitive dependency of the module under test*, and the fix

belonged in the isolation configuration — the same seam that already existed for the parser and metric stack.

4.2 Result and evaluation of usefulness

Pynguin completed successfully and emitted **14 test cases** in ~30 s of search. Coverage of `api.py` + `models.py`, with branch tracking:

Suite	Line	Branch
Manual only (`tests/unit` + `tests/property`)	97.39 %	95.45 %
Generated only (14 raw cases)	57.44 %	9.09 %
Combined	**97.39 %**	**95.45 %**

The generated suite adds zero coverage. Every line and branch it reaches was already covered by the handcrafted tests. That is the honest evaluation required by task 4, and it is what justified aggressive curation rather than wholesale adoption. The branch figure (9.09 %) is the more telling one: search-based generation reached the module's easy surface — constructors and scalar accessors — but almost none of its conditional logic.

4.3 Curation (tasks 5 and 6)

The raw output is preserved verbatim at `pynguin-report/generated-raw-output.py.txt`, deliberately outside `tests/` so pytest does not collect it. The curated survivors live in `tests/generated/test_api_pynguin_curated.py`, which CI runs automatically because the folder sits under `tests/`.

Removed:

What	Why
17 module-constant assertions repeated in *every* test (`module_0.COMMENT == "Comment"` …)	assert unrelated module globals; no oracle for the behaviour under test
`test_case_3/7/9/10/12`	invoked `longname()`, `id()`, `__str__()`, `__ge__()`, `__repr__()` with **no assertion** — proves only "it did not raise"
`test_case_13`	called `create_db("bF4kZ}NM2k0@,", None, None)`, which **writes a SQLite file to the working directory**. A unit test must not touch the filesystem
`test_case_0/4/6/8`	duplicates of existing `tests/unit/` cases, with worse names and random literals

Kept and refactored: two behaviours the manual suite genuinely did not cover, renamed from `test_case_N`, given meaningful locals, and given the assertions Pynguin could not infer:

- `api.open()` on a non-file path raises `UnderstandError` — the manual suite skips `open()` because it needs an on-disk `.udb`, but the guard clause is reachable without one. Generalised into a parametrised test over three non-file paths.
- `Ref` accessors tolerate a partially-populated row — Pynguin built a `Ref` from a mix of `None` and out-of-range integers, a shape no fixture would produce by hand, and that row shape is reachable whenever an analysis pass is interrupted.

Net contribution: 5 tests, 0 % coverage delta, 2 genuinely novel behaviours. That is a modest but real return, and reporting it as modest is the point — the temptation with generated tests is to count them rather than assess them.

4.4 What this says about search-based generation here

Pynguin's DYNAMOSA engine seeds its type system by instrumenting return values and snapshots state with `dill`. Both mechanisms are strained by a database-facade module: the ORM object graph is expensive to serialise, and the API returns ORM-wrapped dataclasses whose construction requires a bound database that the generator has no way to synthesise. That is why it reached constructors and accessors but only 9 % of branches — nearly every interesting branch in `api.py` sits behind a live peewee query.

The practical conclusion is not that the tool is broken, but that its assumptions and this module's design are mismatched. It found two edges worth keeping, and it cost two genuine bugs' worth of diagnosis to get there.

5. Coverage and Mutation Analysis

5.1 Coverage

Coverage is measured with branch tracking on the modules under test (`openunderstand/oudb/api.py` and `models.py`). The large `metric()`/`metrics()` engine and the file/git/info-browser integration methods are explicitly marked `# pragma: no cover`: they require the ANTLR parser and a populated `.udb` database and are therefore out of *unit*-test scope (they belong to integration/oracle testing). This scoping is deliberate and documented, and keeps the coverage figure meaningful for the layer actually under test.

Quality gates enforced by CI: **line ≥ 80 %**, **branch ≥ 70 %**.

Empirical result (Python 3.12.8, `pytest`, branch coverage on `api.py` + `models.py`):

Metric	Result	Gate	Pass?
Tests passed	98 passed, 1	all green	Pass

Metric	Result	Gate	Pass?
	skipped, 3 xfailed (102 collected)		
Line coverage	97.39 % (331 / 339)	≥ 80 %	Pass
Branch coverage	95.45 % (42 / 44)	≥ 70 %	Pass

Per-module: `api.py` 98.98 % line coverage, `models.py` 84.44 % (45 stmts, 7 missed — the ORM `__str__`/`__repr__` dunders). The three `xfail` tests document real faults (Section 7); the one skip is the parser-stage multi-pass concern explained in Section 3.

5.2 Mutation testing

Mutation testing was configured for both mutmut (`setup.cfg`) and Cosmic Ray (`cosmic-ray.toml`). **mutmut has no native Windows support** (mutmut issue #397), so on the Windows development machine the **Cosmic Ray** engine was used instead — this is exactly why the repository ships a cross-platform fallback configuration. The run mutates `openunderstand/oudb/api.py` and executes the full unit suite (`python -m pytest tests -x -q`) against each mutant.

Empirical result (Cosmic Ray, whole-module run over `api.py`):

Quantity	Value
Total mutants	839
Killed	104
Survived	735
Mutation score (killed / total)	**12.40 %**
Survival rate	87.60 %

Interpreting the score

The headline 12.40 % is a **whole-file** score and materially understates the effectiveness of the suite on the code it actually targets. Cosmic Ray mutates *every* statement in the ~1 600-line `api.py`, including the large `metric()`/`metrics()` engine and the file/git/info-browser integration methods. Those regions are **deliberately outside unit-test scope** — they require the ANTLR parser and a populated `.udb` database and are marked `# pragma: no cover` in the coverage configuration (Section 5.1). Mutants planted in code the unit suite never executes **cannot** be killed by that suite, so they inflate the surviving count.

Rather than assert that, it can be measured. Every Cosmic Ray mutant records the source line it was planted on (`mutation_specs.start_pos_row`), and `coverage.json` records which lines the suite executes. Joining the two partitions the run into in-scope and out-of-scope mutants:

Scope	Mutants	Killed	Survived	**Score**

Scope	Mutants	Killed	Survived	**Score**
Whole file (`api.py`)	839	104	735	12.40 %
In-scope (covered lines)	**100**	**75**	**25**	**75.00 %**
Out-of-scope (excluded regions)	739	29	710	3.92 %

These are the figures from the committed session (`cr-session.sqlite`), taken **before** the improvement work below. After the five targeted tests were added, each of those five mutants was re-applied by hand and confirmed to fail the suite, giving an in-scope score of **80 killed / 100 → 80.00 %**. The session file was not regenerated because a full re-run means re-evaluating all 839 mutants (≈ 90 minutes); the command to refresh it is `.\scripts\run_mutation.ps1`.

The interpretation is now empirical rather than rhetorical: **88 % of all mutants sit in code the unit suite is not responsible for**, and on the code it *is* responsible for it kills three out of four. The exact command that reproduces this split is in `docs/TESTING.md` §7, so the number is auditable rather than asserted.

The surviving mutants cluster into three categories:

1. **Out-of-scope integration code (the large majority).** 710 of the 735 survivors — hundreds of `ReplaceComparisonOperator` and `NumberReplacer` mutants in the metric/file/browser methods that unit tests intentionally do not exercise.
2. **Equivalent or unreachable mutants.** `ReplaceComparisonOperator_Eq_Is` / `Eq_IsNot` survivors are semantically equivalent — for comparisons against `None` or interned singletons, `==` and `is` behave identically, so no test can distinguish them. A further cluster of 11 `ReplaceBinaryOperator_BitOr_*` mutants on line 424 is *unreachable by construction*: they mutate the lambda inside `reduce(lambda a, b: a | b, conditions)`, which is never invoked because single-token filters produce a one-element list — and multi-token filters are themselves broken (Finding #4). These are unkillable in principle and correctly *not* counted as test weaknesses.
3. **Genuinely killable survivors in in-scope code.** A small minority in the `Kind`/`Ref`/`Ent`/`Db` accessors that the suite covered but did not assert tightly enough.

Improving the tests accordingly

Category 3 was acted on. Five specific survivors were reproduced by hand, targeted with new assertions, and each fix verified by re-applying the mutation and confirming the suite now fails:

Surviving mutant	Why it survived	Test added
`ents: if refkindstring: →`	every test passed a filter, so	`test_ents_without_a_ref_`

Surviving mutant	Why it survived	Test added
`if not refkindstring:`	the unfiltered path was never taken	`kind_filter_returns_every_kind`
`ents`: `continue` → `break`	only one reference existed, so skipping vs stopping were indistinguishable	`test_ents_skips_non_matching_refs_instead_of_stopping`
`parameters`: `_parent == self._id` → `>=`	only one method had parameters, so a wider comparison matched the same rows	`test_parameters_only_reads_direct_children`
`__eq__`: `==` → `<=`	the one inequality case compared a higher id to a lower one, which `<=` also rejects	property test `test_equality_is_symmetric`
`simplename`: `[-1]` → `[-2]`	no test used a single-segment name, where `[-2]` raises	property test `test_simplename_is_the_last_dotted_segment`

Two of the five were killed by the property tier without any targeted work, which is a concrete argument for property-based testing: Hypothesis generates the boundary inputs (a name with exactly one segment; two ids in the opposite order) that a human writing examples reliably forgets.

Line coverage rose from 97.13 % to 97.39 % and branch coverage from 93.18 % to 95.45 % as a side effect.

Categories 1 and 2 are addressed by scoping and by marking equivalent mutants, not by adding tests. Chasing the bonus **> 85 % mutation score** would require unit-testing the integration engine, which belongs to oracle/integration testing (Section 6) and remains out of scope for this submission.

6. Oracle-Based Validation

Status: not executed. Oracle-based differential testing against the commercial SciTools *Understand* tool was **not performed** for this submission, because a licensed Understand installation was not available on the development machine. No oracle-derived result is claimed anywhere in this report. Differential testing against Understand is listed as a *Bonus Challenge* in the assignment.

Design that was prepared for it. The intended approach uses Understand as the oracle: for a small Java sample, the same queries would be issued through Understand's Python API and through OpenUnderstand, and the resulting entity/reference graphs compared. That would validate semantics pure unit tests cannot reach — e.g. that `Java Call/Java`

`Callby` are produced with the correct scope/ent orientation, and that parent–child containment matches Understand's. Because Understand is licensed and platform-specific, such a stage would run locally rather than in CI. The isolation seams built for the unit suite (`tests/conftest.py`, `tests/_pynguin_support/sitecustomize.py`) are structured so a differential harness can be dropped in where a license is present.

7. Fault Discovery

Building the suite surfaced **eight** faults. The three confirmed, test-backed ones are reproduced by `xfail` tests and written up as complete, ready-to-file GitHub issue reports in `docs/FAULTS.md` (see that file for the issue links once filed against https://github.com/m-zakeri/OpenUnderstand/issues).

1. **`Kind.inv()` always raises `TypeError`.** It calls `inverse.__data__.get("__data__")`, which is `None` (every other call site correctly uses `.__dict__.get("__data__")`), so the inverse of *any* reference kind is unreachable. One-line fix proposed.
2. **`Db.lookup(name)` without a kind filter always returns `[]`** because the final `re.search` builds the literal pattern `java\s+None`. Guarded-fix proposed.
3. **`Ent.refs()` crashes when called with no arguments** (`None.split(",")`) despite the argument being documented as optional; it also contains leftover `print` debug statements.
4. **`Db.ents()` multi-token filtering is contradictory** (ANDs per-token `_kind IN (...)` subqueries that can never be simultaneously satisfied), already flagged in-code with a `TODO`.
5. **`Db.relative_file_name()` can escape the project root.** It uses `os.path.commonprefix`, which compares *characters* rather than path components, so root `com/example` and path `common/Other.java` share the bogus prefix `"com"` and the result becomes `../common/Other.java`; the degenerate case returns `"."` and loses the file name entirely. `os.path.commonpath` is the correct API. **Found by Hypothesis, not by hand.**
6. **`Db.ents()` returns a set where its docstring promises a list**, and `Ent.ents()` returns a list built from a set, so its order is unspecified. This is why CI pins `PYTHONHASHSEED=0`.
7. **`Db.lookup()` interpolates unescaped user input into a regex**, giving a ReDoS surface (`re.escape` is the fix). The SQL query already filters by kind, so the regex is arguably redundant entirely.
8. **`fill.py` never persists the forward kind's inverse.** It assigns to `ref_kind.inverse`, which is not a model field, so `save()` writes nothing. Only the inverse direction of the `_inv` link is ever stored — meaning even after Fault #1 is fixed, `Kind("Java Call").inv()` still cannot reach `Java Callby`. The test fixtures reproduce this asymmetry deliberately.

Each issue includes environment, reproduction steps, expected vs. observed behaviour, the failing test, and a suggested fix for an optional pull request.

8. Lessons Learned

- **Isolation is a design activity.** The single highest-leverage decision was stubbing the parser stack so the api/db layer could be tested as a true unit; it made the suite fast, deterministic, and runnable anywhere — including in CI on three Python versions.
- **Coverage scope must be honest and explicit.** Rather than chase an unreachable 80 % over a 1,600-line facade that embeds an integration-only metric engine, the engine was explicitly excluded from *unit* scope and earmarked for oracle testing. Gaming coverage and declaring scope look similar but are not: the difference is documentation and intent.
- **Tests are excellent bug detectors.** Simply trying to assert the *correct* behaviour of `Kind.inv()` and `Db.lookup()` immediately exposed real defects; `xfail` lets a suite document bugs without going red.
- **Property-based testing finds what examples cannot.** Hand-written examples encode what the author already suspects. Hypothesis falsified an invariant I believed was obviously true — that relativising a path preserves its basename — and produced Fault #5 within seconds. It also killed two mutation survivors for free, and forced an honest correction: my "case-insensitivity" property was itself wrong for Unicode (`'ı'.upper() == 'I'`), which is now documented as a limitation rather than pretended away.
- **A metric is only as good as its scope, and scope can be measured.** The 12.40 % mutation score looked damning until the run was partitioned against coverage data: 88 % of mutants live in code the unit suite is not responsible for, and the in-scope score is 75 %. Computing that split turned an argument into evidence — and the residue it exposed was small enough to actually fix.
- **Tooling reproducibility matters.** Pinned dev requirements, pip caching, `PYTHONHASHSEED=0`, and a Dockerfile make the pipeline reproducible; Pynguin's dependency pins justified a separate environment.
- **"Internal tool bug" deserves a second look.** Pynguin's `NameError: IterableClassWatcher` looked like an unfixable defect in the generator. It was not: the symbol belongs to *GitPython*, a transitive dependency that `api.py` imports for a single out-of-scope function, and `dill` was choking on its deprecated metaclass. Adding `git` to the isolation shim fixed it. The lesson is to read the traceback all the way down to which *package* the failing symbol lives in before concluding the tool is at fault.
- **Automated test generation does not necessarily add value.** This is the most uncomfortable result in the project, and the one I would defend hardest. After the considerable effort of getting Pynguin to run at all — three separate start-up failures,

a dedicated virtual environment, and a custom isolation shim — it produced 14 tests whose measured contribution was **0 % line coverage and 0 % branch coverage** over the handcrafted suite. Not a small gain: *zero*. On its own the generated suite reached only 9 % branch coverage, because nearly every branch in `api.py` sits behind a live peewee query that a search-based generator cannot construct. Worse, the raw output was actively harmful in places: assertion-free calls that prove only "it did not raise", the same 17 irrelevant module-constant assertions pasted into every test, and one case that called `create_db()` and **wrote stray SQLite files into the repository** — 94 of them accumulated across the generation and mutation runs. Curating 14 down to 5 was worth more than adopting all 14. The lesson is not that the tool is bad, but that *tool output is not evidence of test quality*: a suite is worth what it can detect, not what it can be counted as. Had I reported "14 automatically generated tests added" without measuring the delta, the number would have been true and the claim misleading.

9. Deliverables and Where to Find Them

Deliverable	Location in repository
Unit tests (handcrafted)	`tests/unit/` (`test_kind.py`, `test_ref.py`, `test_entity_and_db.py`, `test_misc.py`)
Property-based tests (bonus)	`tests/property/test_api_properties.py`
Generated tests (curated)	`tests/generated/test_api_pynguin_curated.py`
Generated tests (raw Pynguin output)	`pynguin-report/generated-raw-output.py.txt`
Test fixtures / isolation	`tests/_isolation.py`, `tests/conftest.py`, `tests/_pynguin_support/sitecustomize.py`
CI/CD pipeline	`.github/workflows/ci.yml`
Coverage reports	`coverage.xml`, `coverage.json`, `htmlcov/index.html`, `reports/junit.xml`
Mutation config	`cosmic-ray.toml` (Cosmic Ray), `setup.cfg` (mutmut)
Pynguin scripts	`scripts/run_pynguin.sh` / `.ps1`
Mutation scripts	`scripts/run_mutation.sh` / `.ps1`
Fault reports (GitHub issues)	`docs/FAULTS.md`
Testing / reproduce guide	`docs/TESTING.md`
This report	`docs/REPORT.md` (source), `docs/OpenUnderstand-HW3-Final-Report.docx`, `docs/OpenUnderstand-HW3-Final-Report.pdf`

Deliverable	Location in repository
Reproducible container	`Dockerfile`, `.dockerignore`

Appendix A — How to reproduce

Any of Python 3.10, 3.11 or 3.12 works (CI tests all three). Every command is a single line, so it can be pasted into PowerShell, `cmd` or a POSIX shell without edits. Full step-by-step instructions are in `docs/TESTING.md`.

Windows (PowerShell), from the repo root:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
ruff check tests
pytest tests --cov=openunderstand.oudb.api --cov=openunderstand.oudb.models --cov-branch --cov-report=term-missing --cov-report=html --cov-report=json
.\scripts\run_mutation.ps1
```

Steps 4–5 above are the lint gate (Parts 1–2) and the unit tests plus coverage (Parts 3 & 5); step 6 is mutation testing (Part 5, Cosmic Ray — mutmut has no native Windows support).

Linux/macOS: identical, except `source .venv/bin/activate` and `./scripts/run_mutation.sh` (mutmut).

Pynguin (Part 4) needs its own environment because it pins older dependencies:

```
python -m venv .venv-pynguin
.\.venv-pynguin\Scripts\Activate.ps1
pip install "pynguin>=0.43.0" -r requirements.txt
.\scripts\run_pynguin.ps1
```

See `docs/TESTING.md` for the full guide, including the CI job breakdown and the Docker workflow.